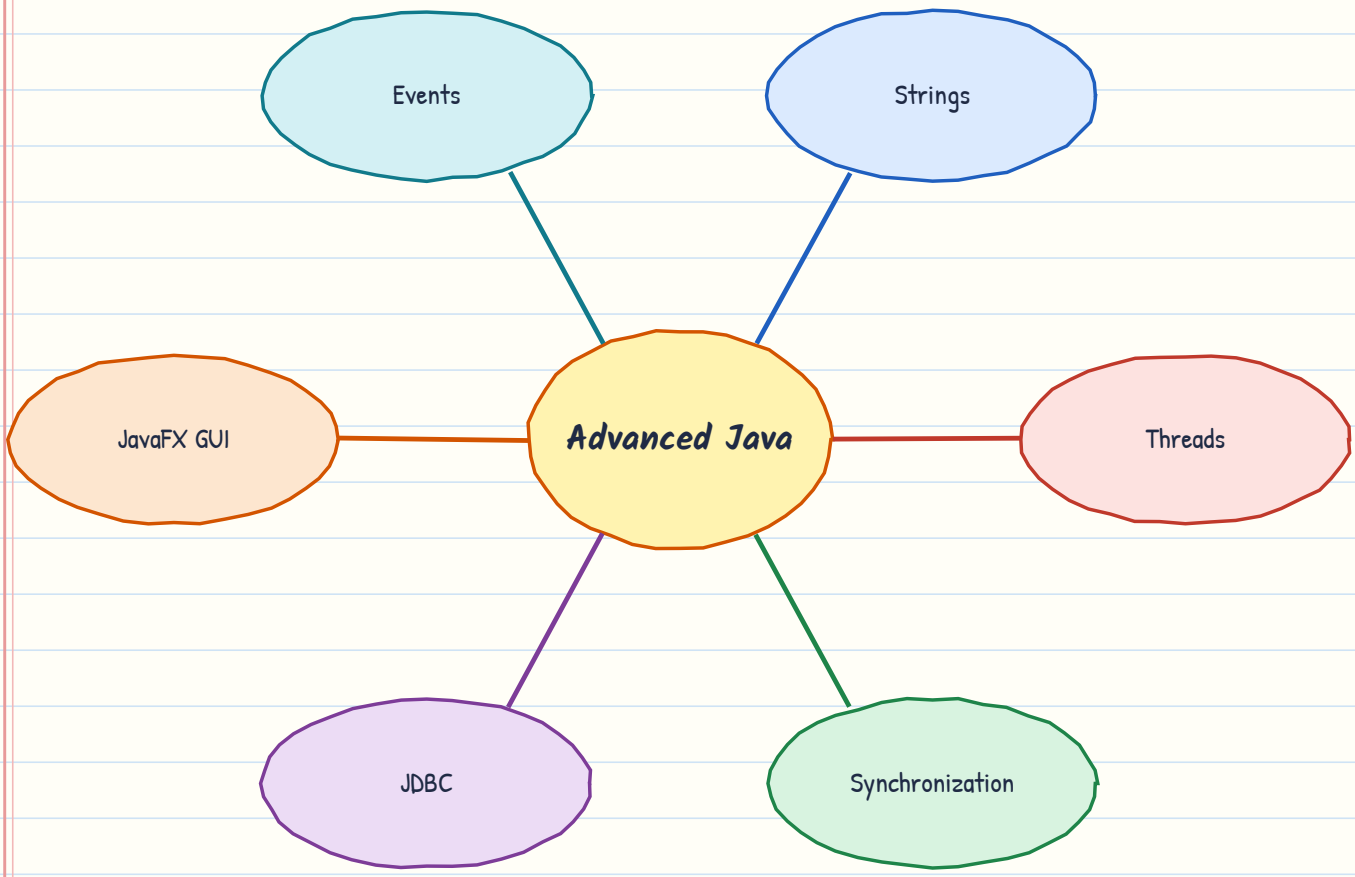
UNIT V

Strings, Multithreading, JDBC, JavaFX

String & StringBuffer | Thread life cycle | Synchronization | JDBC | GUI



Mind map: everything you will learn in this unit

How to use these notes

Tip: read the diagrams first, then the text. Try writing each example yourself - code you type sticks in memory, code you only read does not.

I. String Handling

A ***String*** is a sequence of characters. In Java it is an ***object*** of class **`String`**, and it is ***immutable*** – once created it can never be changed (every "change" makes a new String).

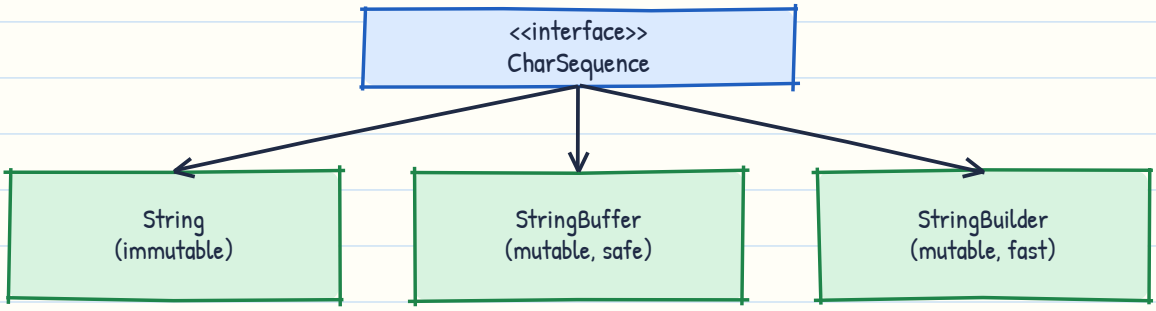


Fig 1: Interface CharSequence and its implementers

Creating strings & the string pool

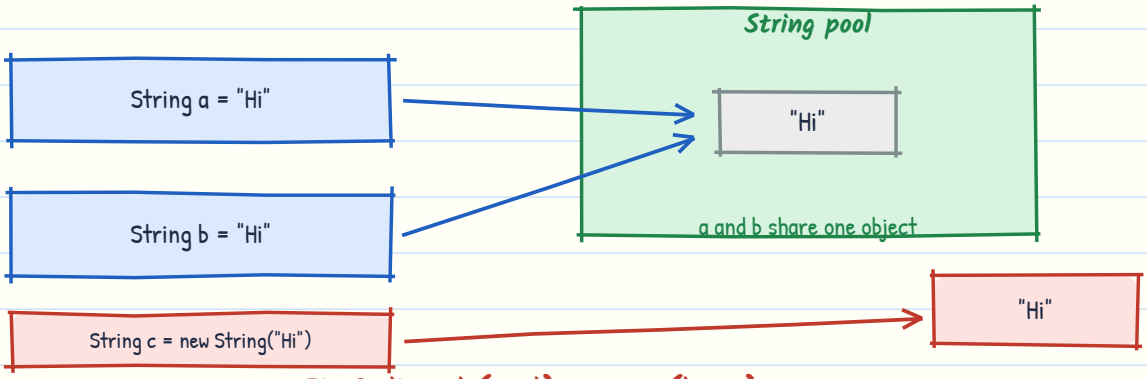


Fig 2: literal (pool) vs new (heap)

```
String s1 = "Java";                 // literal -> pool
String s2 = new String("Java");     // new object in heap
System.out.println(s1 == s2);       // false  (different objects)
System.out.println(s1.equals(s2));  // true   (same content)
```

Classic exam trap

compares REFERENCES, equals() compares CONTENT. Always use equals() for string

Extracting characters

Method	Purpose	Example ("HELLO")
length()	number of chars	5
charAt(i)	char at index	charAt(1) = E
toCharArray()	to char[]	[H,E,L,L,O]
getChars(...)	copy part into char[]	-

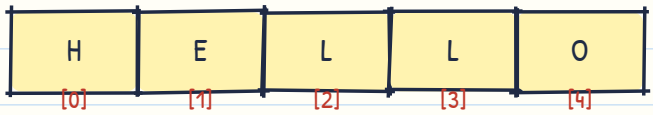


Fig 3: index positions of "HELLO"

Comparison

Method	Meaning
equals(s)	exact content match (case-sensitive)
equalsIgnoreCase(s)	match ignoring case
compareTo(s)	0 if equal, <0 if smaller, >0 if bigger (dictionary order)
startsWith / endsWith	prefix / suffix test

Modifying & searching

```
String s = "  Hello World  ";
s.trim();                          // "Hello World"
s.toUpperCase();  s.toLowerCase();
"Hello".substring(1, 4);           // "ell"   (start incl., end excl.)
"Hello".concat(" Java");           // "Hello Java"
"banana".replace('a', 'o');        // "bonono"
"Hello".indexOf('l');              // 2  (first)
"Hello".lastIndexOf('l');          // 3
"Hello".contains("ell");           // true
"a,b,c".split(",");                // {"a","b","c"}
```

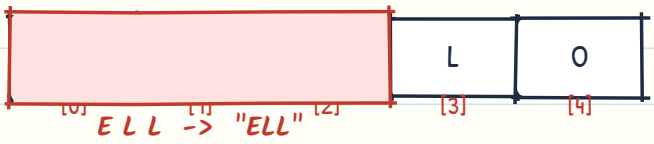


Fig 4: substring(1,4) on "HELLO"

Class StringBuffer (mutable string)

Use when you modify text often - it changes the *same object* instead of creating new ones.

`StringBuilder` is the same but not thread-safe (faster).

```java
StringBuffer sb = new StringBuffer("Hello");
sb.append(" World");        // Hello World
sb.insert(0, ">> ");        // >> Hello World
sb.reverse();               // dlroW olleH >>
sb.delete(0, 3);  sb.replace(0, 1, "D");
sb.length();  sb.capacity();  // default capacity 16
```

	String	StringBuffer
Mutable?	No	Yes
Thread-safe	yes (immutable)	yes (synchronized)
Speed (many edits)	slow	fast

2. Multithreaded Programming

A ***thread*** is a lightweight path of execution inside a program. ***Multithreading*** runs many threads at once so work overlaps - e.g. a music player plays audio while the UI stays responsive.

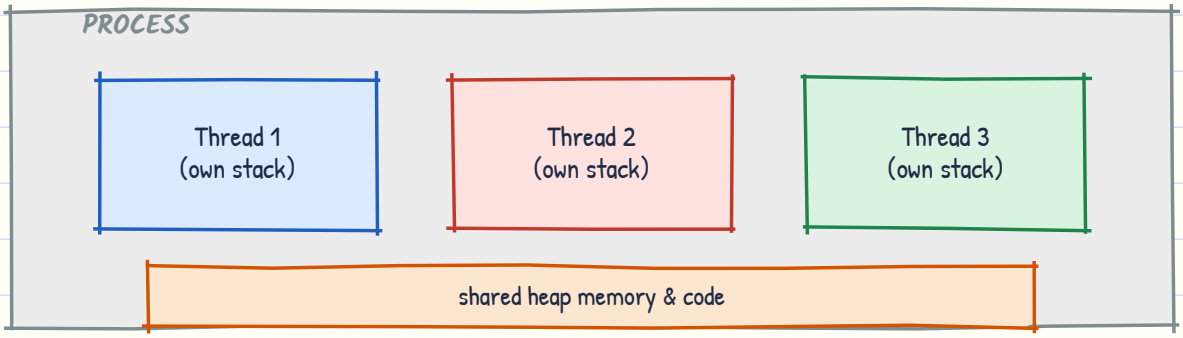


Fig 5: One process, many threads (sharing memory)

- ***Need:*** better CPU use, responsive apps, faster results on ***multi-core*** processors (each core runs a thread truly in parallel).
- ***Process*** = heavy, own memory | ***Thread*** = light, shares memory.

Main thread & creating threads

Every program starts with the ***main thread***. Two ways to create new threads:

```java
class A extends Thread {                        // way 1: extend Thread
    public void run() { System.out.println("Thread A running"); }
}
class B implements Runnable {                   // way 2: implement Runnable (preferred)
    public void run() { System.out.println("Thread B running"); }
}
public class Demo {
    public static void main(String[] args) {
        new A().start();                        // start(), NOT run()!
        new Thread(new B()).start();
        System.out.println(Thread.currentThread().getName());   // main
    }
}
```

Remember

start() creates a NEW thread which calls run(). Calling run() directly is just a normal method call - no new thread!

Thread states (life cycle)

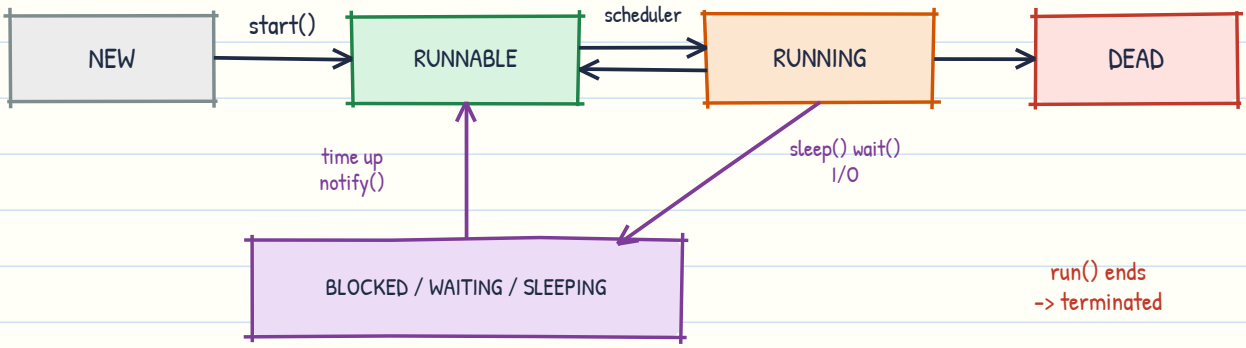


Fig 6: Life cycle of a thread

Thread priority & useful methods

Method	Purpose
start()	begin new thread
sleep(ms)	pause for time
join()	wait for another thread to finish
setPriority(1-10)	hint to scheduler (MIN=1, NORM=5, MAX=10)
isAlive(), getName()	status / name

Race condition, synchronization & deadlock

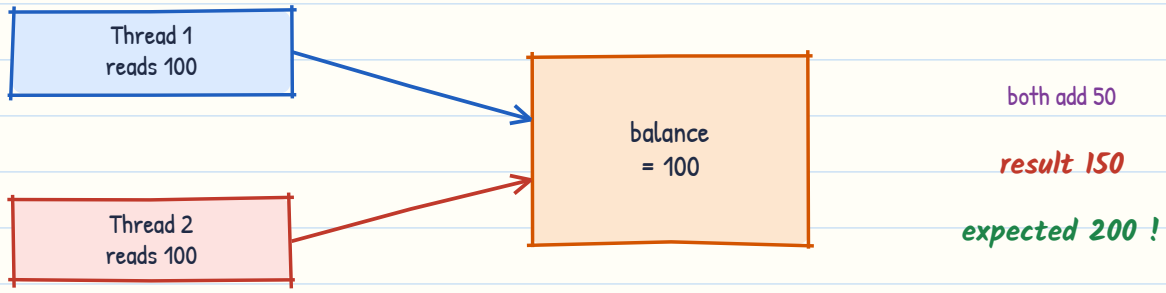
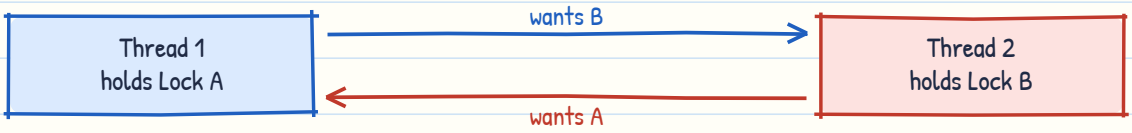


Fig 7: Race condition - two threads update the same data

Synchronization lets only ***one thread at a time*** enter critical code using the `synchronized` keyword (a lock / monitor).

```
class Counter {
    int c = 0;
    synchronized void inc() { c++; }      // one thread at a time
    void other() { synchronized (this) { c--; } }   // block version
}
```



both wait forever = DEADLOCK

Fig 8: Deadlock - each thread waits for the other's lock

Inter-thread communication

Threads cooperate with `wait()`, `notify()`, `notifyAll()` (called inside synchronized code), e.g. ***producer-consumer***: consumer waits till producer has data.



Fig 9: Producer - Consumer

Suspend / Resume / Stop

suspend(), resume(), stop() are DEPRECATED (unsafe, may cause deadlock). Instead use a boolean flag or wait()/notify() to pause, resume and stop threads.

3. Java Database Connectivity (JDBC)

JDBC is the Java API for connecting to a relational database (MySQL) and running SQL from a program.

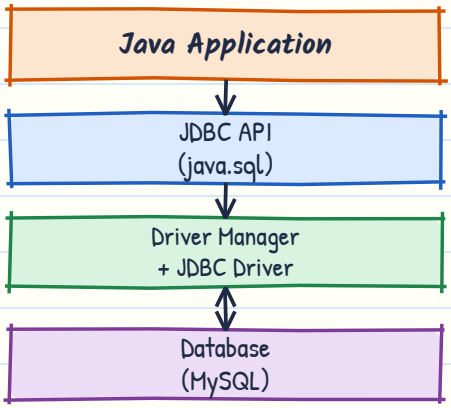


Fig 10: JDBC architecture

Setup steps

- Install MySQL and create a database/table.
- Download **MySQL Connector/J** (`mysql-connector-j.jar`) and add it to the **classpath**.
- Write the program using the 5 JDBC steps below.

Step	Code
1. Load driver	Class.forName("com.mysql.cj.jdbc.Driver")
2. Connect	DriverManager.getConnection(url,user,pass)
3. Statement	con.createStatement()
4. Execute	executeQuery() / executeUpdate()
5. Close	con.close()

```java
import java.sql.*;
public class Db {
    public static void main(String[] a) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/college", "root", "password");
        Statement st = con.createStatement();
        ResultSet rs = st.executeQuery("SELECT id, name FROM student");
        while (rs.next())                                   // move row by row
            System.out.println(rs.getInt("id") + " " + rs.getString("name"));
        con.close();
    }
}
```

```
> OUTPUT
101 Asha
102 Ravi
```

ResultSet interface

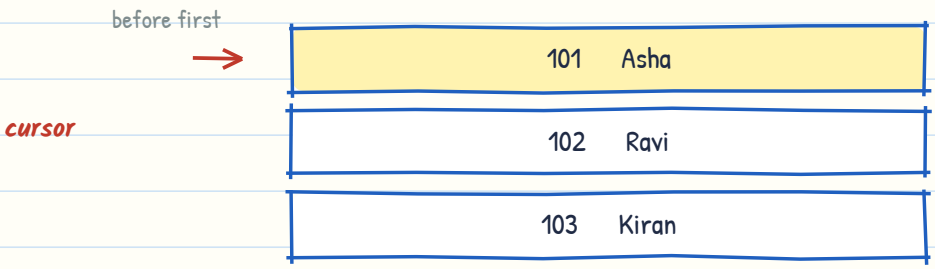


Fig 11: ResultSet cursor moves with next()

Method	Purpose
next()	move to next row; false when no more rows
getInt/getString(col)	read column value
executeUpdate()	INSERT / UPDATE / DELETE, returns rows affected

```java
PreparedStatement ps = con.prepareStatement("INSERT INTO student VALUES(?,?)");
ps.setInt(1, 104);  ps.setString(2, "Meena");
ps.executeUpdate();            // safer than string concatenation (stops SQL injection)
```

4. JavaFX GUI

JavaFX is Java's modern library for desktop GUI apps. **Scene Builder** is a drag-and-drop tool that creates the layout as an **FXML** file.

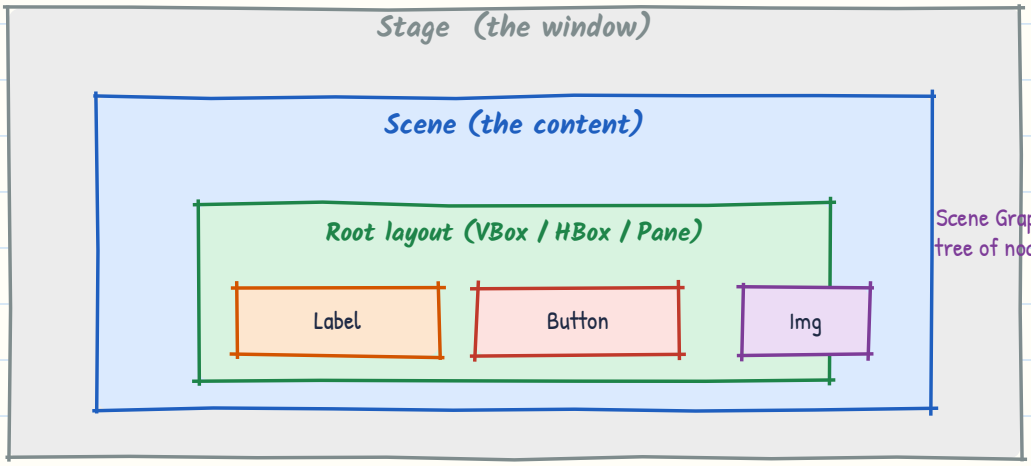


Fig 12: JavaFX app window structure

```
import javafx.application.Application;
import javafx.scene.*;  import javafx.scene.control.*;  import javafx.scene.layout.*;
import javafx.stage.Stage;
public class Hello extends Application {
    public void start(Stage stage) {                    // entry point of JavaFX
        Label lbl = new Label("Hello JavaFX");
        Button btn = new Button("Click me");
        btn.setOnAction(e -> lbl.setText("Button clicked!"));   // event handling
        VBox root = new VBox(10, lbl, btn);                 // layout
        stage.setScene(new Scene(root, 300, 200));
        stage.setTitle("My App");  stage.show();
    }
    public static void main(String[] a) { launch(a); }
}
```

Displaying text & images

```
Text t = new Text("Welcome");  t.setFont(Font.font("Arial", 24));
ImageView iv = new ImageView(new Image("file:logo.png"));
iv.setFitWidth(100);
```

Laying out nodes in the scene graph

Layout	Arranges nodes
HBox	in a horizontal row
VBox	in a vertical column
BorderPane	top, bottom, left, right, center
GridPane	rows and columns grid

StackPane	on top of each other
FlowPane	wrapping flow

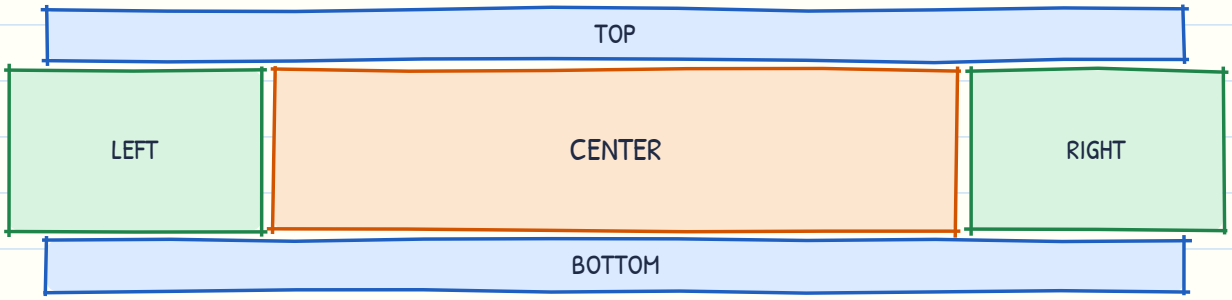


Fig 13: BorderPane regions

Event handling & mouse events

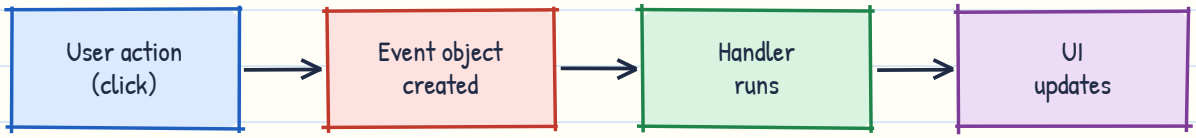


Fig 14: Event flow

```
btn.setOnAction(e -> System.out.println("clicked"));        // button
pane.setOnMouseClicked(e -> System.out.println(e.getX() + "," + e.getY()));
pane.setOnMouseEntered(e -> pane.setStyle("-fx-background-color: yellow;"));
pane.setOnMouseExited(e -> pane.setStyle(""));
```

Mouse event	When
setOnMouseClicked	click
setOnMousePressed / Released	button down / up
setOnMouseMoved / Dragged	cursor moves / drags
setOnMouseEntered / Exited	cursor enters / leaves node

Quick Revision & Exam Questions

Topic	Memory line
String	immutable; == vs equals(); pool
StringBuffer	mutable; append, insert, reverse
Thread create	extends Thread or implements Runnable; start()
States	New - Runnable - Running - Blocked - Dead
synchronized	one thread at a time; prevents race condition
JDBC steps	driver, connect, statement, execute, close

JavaFX	Stage > Scene > layout > nodes

Likely exam questions

- Explain String methods for extraction, comparison, modification and searching.
- Differentiate String and StringBuffer.
- What is multithreading? Explain thread life cycle with a diagram.
- Explain ways of creating threads with programs.
- Explain synchronization, race condition and deadlock.
- Explain JDBC architecture and the steps to connect to MySQL.
- Explain JavaFX window structure; write a program with a button and event handling.